Big Data Analytics — Platform & Execution Guide (All 11 Practicals)

Quick reference: where each practical runs, which interface/editor you actually type code into, and the exact sequence of steps. Based directly on your lab journal.

Practical 1 — Install, Configure & Run Hadoop/HDFS

Platform: Windows OS (native install, no editor needed for "code")
Interface used: Command Prompt (CMD) + Notepad/VS Code (for XML configs) + Control Panel (for env vars) + Browser (for web UI)

How it's done:
1. Install Java JDK 8 → note install path.
2. Download Hadoop 3.2.1 binary, extract with 7-Zip in **two steps** (`.tar.gz` → `.tar` → folder).
3. Set env vars via **Control Panel → System → Advanced System Settings → Environment Variables**: `JAVA_HOME`, `HADOOP_HOME`, and add both `\bin` and `\sbin` to `PATH`.
4. Open `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml` (inside `%HADOOP_HOME%\etc\hadoop`) in **Notepad or VS Code** and paste the config blocks.
5. Back in CMD: `hdfs namenode -format`
6. Start services: `start-dfs.cmd` then `start-yarn.cmd`
7. Verify in a **browser**: `localhost:9870` (HDFS NameNode UI), `localhost:8088` (YARN ResourceManager UI)
8. Run `hdfs dfs -ls /` and `hdfs dfs -mkdir /user` in CMD.

There's no "code editor" here in the programming sense — it's installation + config-file editing + CMD commands.

Practical 2 — Word Count (Apache Pig)

Platform: Apache Pig, running in **local mode** (does *not* need Hadoop daemons running)
Interface used: CMD → Pig **Grunt shell**

How it's done:
1. Create the input text file in **Notepad**, save to Desktop (e.g. `loadwordscount.txt`).
2. Open CMD, type: `pig -x local` → this drops you into the `grunt>` prompt.
3. Type each Pig Latin line directly at `grunt>` (LOAD → FOREACH/TOKENIZE → GROUP → COUNT → DUMP).
4. Output prints right there in the same shell after each `DUMP`.

No separate script file is used — it's typed interactively, line by line, into Grunt.

Practical 3 — Weather Dataset (Apache Pig)

Platform & interface: Identical to Practical 2 — Apache Pig, `pig -x local`, Grunt shell.

How it's done:
1. Prepare `weather.csv` (Notepad/Excel) with columns Year, Month, Day, Station, MaxTemp, MinTemp.
2. Launch `pig -x local`.
3. Type the LOAD → GROUP BY year → FOREACH with MAX/MIN/AVG → DUMP commands at `grunt>`.

Practical 4 — MongoDB + Python

Platform: MongoDB Community Server (the `mongod` service) + Python
Interface used: Two CMD windows (one running the Mongo server, one for the shell) + a **Python editor/runner** for the `.py` scripts

How it's done:
1. Install MongoDB (MSI installer), create data directory `C:\data\db`.
2. **CMD window 1:** run `mongod` → leave this running as the server.
3. **CMD window 2:** run `mongo` → this opens the **Mongo shell**, used later for verification (`show dbs`, `db.student.find()`, etc.).
4. Install PyMongo: `pip install pymongo`.
5. Write the named scripts — `create_db.py`, `create_collection.py`, `insert_one.py`, `insert_many.py` — in a Python editor (VS Code/Notepad++/IDLE all work) and run each with `python scriptname.py` from CMD.
   - *Note:* the journal also shows a Jupyter-Notebook-style (`In [1]:` cell) example for bulk insert/aggregation — Jupyter is a perfectly fine alternative if you prefer running cells instead of standalone files.
6. Cross-check the inserted data using the **Mongo shell** from step 3.

Practical 5 — MongoDB App Re-implemented in Pig

Platform: Apache Pig, **local mode** — same as Practicals 2 & 3
Interface used: CMD → Pig Grunt shell

How it's done:
1. Prepare `students.csv` (name, address) in Notepad.
2. `pig -x local` to enter Grunt.
3. LOAD the CSV → GROUP BY address (city) → COUNT → DUMP/STORE.

Note: the *Aim* mentions registering MongoDB connector JARs (`mongo-hadoop-pig`, `mongo-java-driver`), but the script actually executed just reads a local CSV with `PigStorage` — no live MongoDB connection is really used in the run shown.

Practical 6 — Hive (Student App)

Platform: Apache Hive — this one **does need Hadoop running**, because Hive's metastore/warehouse sits on HDFS.
Interface used: CMD → **Hive CLI** (`hive>` prompt) + Notepad/VS Code for `hive-site.xml`

How it's done:
1. Download Hive, extract, set `HIVE_HOME` env var, add `\bin` to PATH.
2. Edit `hive-site.xml` (Derby metastore config) in Notepad/VS Code.
3. **Start Hadoop first:** `start-dfs.cmd`, `start-yarn.cmd` (Practical 1's services).
4. In CMD, type `hive` → drops you into the `hive>` prompt.
5. Type HiveQL directly there: `CREATE DATABASE`, `CREATE TABLE`, `LOAD DATA LOCAL INPATH ...`, `SELECT`, `GROUP BY`.

This is the one Pig-adjacent practical where Hadoop daemons genuinely need to be up first.

Practical 7 — JAQL Simulation

Platform: Python 3.x
Interface used: **Jupyter Notebook** (explicitly listed in Software Required)

How it's done:
1. Open Jupyter Notebook (`jupyter notebook` from CMD/Anaconda, or via Anaconda Navigator).
2. Write the code (using the built-in `json` module + list comprehensions for filter/transform/group/aggregate) into notebook cells.
3. Run cells with Shift+Enter; output prints below each cell.

Practical 8 — Decision Tree Classification

Platform: R
Interface used: **RStudio** (script editor pane + Console + Plots pane)

How it's done:
1. Open RStudio, write/paste the script into a new `.R` file in the Source editor.
2. `install.packages("caTools")`, `install.packages("rpart")`, `install.packages("ElemStatLearn")` (run once).
3. Run the script (Ctrl+Enter line-by-line, or "Run" for the whole file).
4. Console shows the confusion matrix; **Plots pane** shows the decision boundary visualizations and the tree diagram (`plot(classifier); text(classifier)`).

Practical 9 — SVM Classification

Platform & interface: Same as Practical 8 — R / RStudio.

How it's done:
1. Same dataset (`Social_Network_Ads.csv`), same RStudio script-editor workflow.
2. Additional package: `install.packages("e1071")` for `svm()`.
3. Console → model summary + confusion matrix; Plots pane → classification boundary plots for train/test sets.

Practical 10 — Multiple Regression

Platform & interface: R / RStudio.

How it's done:
1. Dataset is pulled **directly from a URL** with `read.csv("https://raw.githubusercontent.com/...")` — RStudio needs internet access for this one (not a local file).
2. `install.packages("caTools")` → split data → `lm()` to fit the model → `predict()`.
3. Console shows predictions/confusion table; Plots pane shows the `cdplot()` conditional density plots for gpa/gre/rank.

Practical 11 — Cluster Visualization (K-Means)

Platform & interface: R / RStudio.

How it's done:
1. Dataset loaded from a **local file path** (`read.csv('F:\\GitHub\\...\\Mall_Customers.csv')`) — update this path to wherever your CSV actually sits on the lab PC.
2. Run the elbow-method loop (`kmeans()` for i = 1 to 10) → plot WCSS to pick cluster count.
3. Fit final `kmeans()` with 5 centers, then `library(cluster)` → `clusplot()` to render the cluster plot in the Plots pane.

At-a-Glance Summary Table

#	Practical	Platform	Interface / "Editor"
1	Hadoop/HDFS install	Windows	CMD + Notepad (XML configs)
2	Word Count	Apache Pig	Grunt shell (`pig -x local`)
3	Weather data	Apache Pig	Grunt shell
4	MongoDB + Python	MongoDB + Python	Mongo shell + `.py` files (CMD/VS Code/Jupyter)
5	Mongo app in Pig	Apache Pig	Grunt shell
6	Hive	Apache Hive (needs Hadoop up)	Hive CLI (`hive>`)
7	JAQL	Python	Jupyter Notebook
8	Decision Tree	R	RStudio
9	SVM	R	RStudio
10	Multiple Regression	R	RStudio
11	Clustering	R	RStudio

Bottom line for exam day: four distinct environments to be ready to open: **CMD** (Hadoop/Pig/Hive), **Mongo shell + Python editor**, **Jupyter Notebook**, and **RStudio**. Only Practical 1 (obviously) and Practical 6 (Hive) actually require Hadoop's daemons to be running.